OLTP vs OLAP

為什麼需要兩種不同的資料庫

想像你在設計一個電商平台。系統裡同時存在兩種截然不同的需求：

一種是用戶下單——你需要在幾毫秒內完成一筆事務：扣庫存、建訂單、記錄付款。這個操作每秒可能發生幾千次，每次只動幾行資料，必須保證 ACID 一致性，任何一步失敗都要回滾。

另一種是業務分析——財務團隊想知道「上個月每個地區的銷售額，按商品類別分組，只看客單價超過 500 元的訂單」。這個查詢要掃描幾億筆歷史訂單，做大量的聚合運算，可能跑 30 秒。一天只跑幾次，不需要即時一致性，但需要掃描海量的資料。

這兩種需求在**幾乎每個維度上都相反**。用同一個資料庫同時滿足兩者，你會兩頭都做不好：分析查詢掃全表會鎖住業務資料，業務寫入的大量隨機 I/O 又讓分析查詢的效率低落。

這就是 **OLTP（Online Transactional Processing）** 和 **OLAP（Online Analytical Processing）** 分開存在的根本原因——它們解決的是兩種完全不同的問題，底層的儲存引擎設計也因此走向了完全不同的方向。

buildpoat.org

OLTP：為交易而生

OLTP 系統的設計哲學是：**對少量資料做快速、精準的讀寫，同時保證資料的一致性**。

列式儲存（Row-oriented Storage）

OLTP 資料庫把資料按行（row）儲存。一筆訂單的所有欄位——訂單 ID、用戶 ID、金額、狀態、時間戳記——連續存在磁碟上的同一個位置。

```
磁碟上的實際排列：
[order_id=1, user_id=101, amount=500, status="paid", created_at="2024-01-01"]
[order_id=2, user_id=203, amount=1200, status="pending", created_at="2024-01-01"]
[order_id=3, user_id=101, amount=300, status="paid", created_at="2024-01-02"]
```

這個設計對交易操作非常友好：

- **讀取一筆訂單**：找到那一行，一次 I/O 就拿到所有欄位
- **更新訂單狀態**：定位到那一行，修改其中一個欄位
- **插入新訂單**：在末尾附加一行

列式儲存對「點查詢（point query）」和「小範圍更新」效率極高，這正是業務交易的典型模式。

索引是 OLTP 的靈魂

OLTP 資料庫靠索引把查詢從「掃全表」變成「直接跳到那幾行」。

```
-- 沒有索引：掃描所有訂單才能找到用戶 101 的訂單
SELECT * FROM orders WHERE user_id = 101;

-- 有 user_id 索引：直接定位，O(log n) 而不是 O(n)
CREATE INDEX idx_orders_user_id ON orders(user_id);
```

B-tree 索引是最常見的結構，讓等值查詢和範圍查詢都能高效執行。OLTP 的設計核心之一，就是為業務的查詢模式建立正確的索引。

ACID 事務

OLTP 最核心的承諾是 ACID：

- **Atomicity（原子性）**：「扣庫存 + 建訂單 + 記付款」三個操作，要嘛全成功，要嘛全回滾，不會出現庫存扣了但訂單沒建的狀態
- **Consistency（一致性）**：事務前後，資料庫的約束條件（外鍵、唯一鍵、check constraint）都必須滿足
- **Isolation（隔離性）**：兩筆並發的事務互不干擾，不會看到彼此的中間狀態
- **Durability（持久性）**：事務一旦 commit，即使伺服器立刻崩潰，資料也不會遺失（靠 WAL 實現）

OLTP 的典型代表

PostgreSQL、MySQL、Oracle 是傳統的 OLTP 關聯式資料庫。DynamoDB、MongoDB 雖然是 NoSQL，在交易操作模式上仍屬於 OLTP 的範疇——它們都是為高並發的點查詢和小量寫入設計的。

OLTP 的效能基準（以 PostgreSQL 為例）：

buildmoat.org

- 簡單的主鍵查詢：數萬 QPS / 核心
- 帶索引的範圍查詢：數千 QPS / 核心
- 複雜的多表 JOIN：數百 QPS / 核心
- 最適合的資料量：單表幾千萬行以內，超過開始需要分片

OLAP：為分析而生

OLAP 系統的設計哲學是：**對海量歷史資料做高效的聚合運算，即使單個查詢需要掃描幾億行**。

行式儲存（Column-oriented Storage）

OLAP 資料庫把資料按欄（column）儲存。所有訂單的金額存在一起，所有訂單的狀態存在一起。

```
磁碟上的實際排列：
[amount]   : 500, 1200, 300, 800, 2100, ...（所有訂單的金額連續排列）
[status]   : "paid", "pending", "paid", "paid", "cancelled", ...
[user_id]  : 101, 203, 101, 405, 101, ...
[created_at]: "2024-01-01", "2024-01-01", "2024-01-02", ...
```

為什麼這對分析查詢更好？考慮這個查詢：

```
SELECT region, SUM(amount)
FROM orders
WHERE created_at >= '2024-01-01'
GROUP BY region;
```

這個查詢只需要 `amount`、`created_at`、`region` 三個欄位，完全不需要其他欄位（用戶 ID、商品 ID、收件地址……）。

- **列式儲存**：必須讀取每一行的所有欄位，但實際只用到 3 個欄位，大量的 I/O 都浪費了
- **行式儲存**：只讀取這 3 個欄位對應的資料塊，I/O 量可能少了 90% 以上

壓縮：行式儲存的天然優勢

同一個欄位的資料類型相同、值域相近，壓縮率遠高於列式儲存。

buildmoat.org

`status` 欄位裡只有 "paid"、"pending"、"cancelled" 幾個值，用字典編碼可以把字串壓縮成 2 位元。幾億筆訂單的 status 欄位，壓縮後可能只有幾 MB。

```
字典編碼：
  paid      → 0
  pending   → 1
  cancelled → 2

實際儲存：0, 1, 0, 0, 2, 0, 0, 1, ...  （每個值只需 2 bits）
```

壓縮帶來雙重收益：儲存空間小，讀取的資料量少，I/O 速度更快。

向量化執行（Vectorized Execution）

現代 OLAP 引擎不是一行一行地處理資料，而是一次處理一個「批次（batch）」——通常是 1024 或 4096 行。這讓 CPU 能充分利用 SIMD 指令，對整個向量做並行運算：

```
傳統逐行執行：
  for row in rows:
      if row.amount > 500:
          sum += row.amount   # 每次一行，CPU 指令開銷大

向量化執行：
  amounts = load_column("amount")  # 一次載入 1024 個 amount
值
  mask = amounts > 500             # SIMD：一條指令比較 1024
個值
  result = sum(amounts[mask])      # SIMD：一條指令加總
```

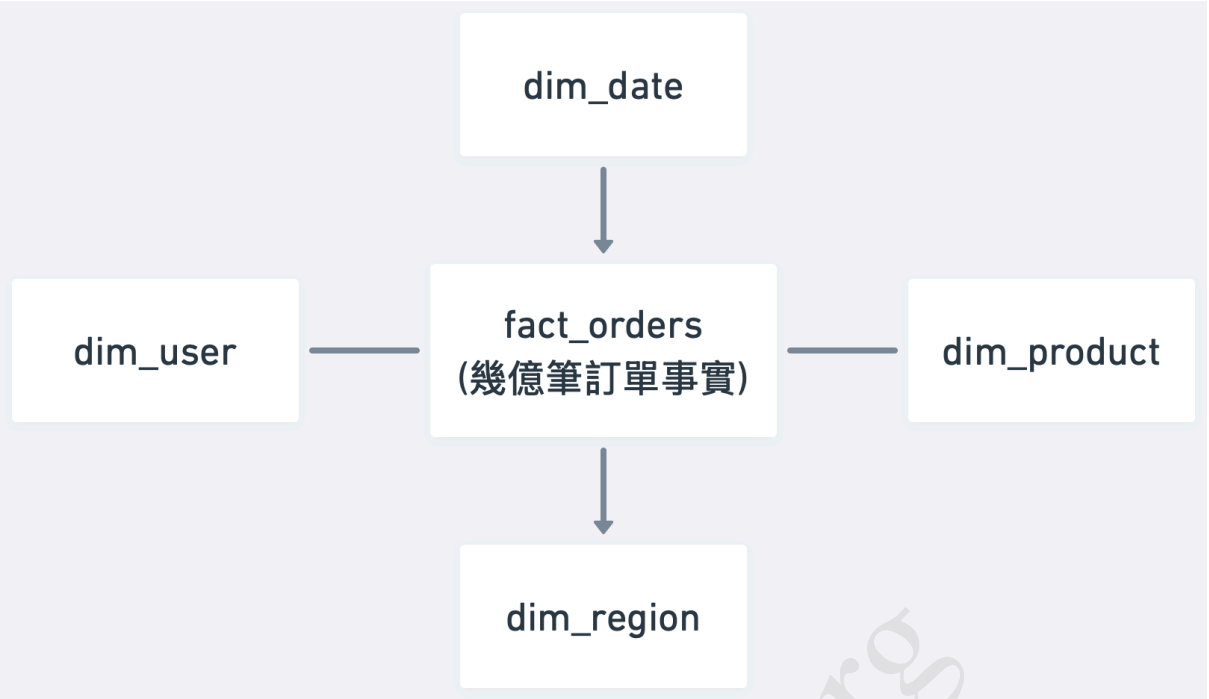
這讓 OLAP 查詢的吞吐量比逐行執行高出幾個數量級。

資料倉儲的 Schema 設計

OLTP 資料庫強調正規化（normalization）——把資料拆成多個表，用外鍵關聯，避免冗餘。這在交易時很好，但在分析時要 JOIN 幾十個表，效率很差。

OLAP 資料倉儲通常用**星型 Schema（Star Schema）**：

buildmoat.org



中心是**事實表（fact table）**，存放每一筆業務事件（訂單、點擊、付款），通常有幾億到幾兆行。周圍是**維度表（dimension table）**，存放描述性的屬性（用戶資料、商品資料、地區、時間）。

分析查詢就是：從事實表做聚合，用維度表做篩選和分組。這個結構比高度正規化的 OLTP schema 更適合大規模 JOIN 和聚合。

OLAP 的典型代表

雲端資料倉儲：BigQuery（Google）、Redshift（AWS）、Snowflake——完全管理的服務，按查詢量或儲存量計費，適合大多數公司的 BI 需求。

自建 OLAP：ClickHouse——開源，行式儲存，擅長高並發的即時分析查詢，效能極高；Apache Doris——同樣開源，支援即時更新。

嵌入式 OLAP：DuckDB——可以直接在 Python 進程裡跑分析查詢，不需要獨立的資料庫服務，適合資料科學和本地分析。

```
-- BigQuery 的典型分析查詢：幾秒內掃描幾億行
SELECT
    d.region_name,
    p.category,
    SUM(f.amount) AS total_revenue,
    COUNT(*) AS order_count,
    AVG(f.amount) AS avg_order_value
FROM fact_orders f
```

buildinpat.org

```
JOIN dim_region d ON f.region_id = d.region_id
JOIN dim_product p ON f.product_id = p.product_id
WHERE f.created_at BETWEEN '2024-01-01' AND '2024-12-31'
  AND f.status = 'paid'
GROUP BY d.region_name, p.category
ORDER BY total_revenue DESC;
```

OLTP vs OLAP：核心差異

維度	OLTP	OLAP
設計目標	快速的個別交易	大規模聚合分析
查詢模式	點查詢、小範圍讀寫	全表掃描、聚合、GROUP BY
資料量	單次操作幾行到幾百行	單次查詢幾億到幾兆行
並發數	高（數千個並發用戶）	低（幾十個分析師）
查詢延遲	幾毫秒到幾十毫秒	幾秒到幾分鐘
儲存結構	列式（row-oriented）	行式（column-oriented）
Schema	高度正規化	星型 / 雪花型 Schema
資料新鮮度	即時（毫秒級）	通常有延遲（分鐘到小時）
代表系統	PostgreSQL、MySQL、DynamoDB	BigQuery、Snowflake、ClickHouse
優化手段	B-tree 索引、事務、WAL	行式壓縮、向量化執行、分區裁剪

buildmoat.org

如何連接 OLTP 和 OLAP

在實際系統裡，業務資料先進入 OLTP，然後搬到 OLAP 供分析。連接兩者的管線有幾種做法。

ETL（Extract, Transform, Load）

最傳統的做法。定期（例如每天凌晨）從 OLTP 資料庫抽取資料，做清洗和轉換，再載入 OLAP 倉儲。

```
PostgreSQL ──────→ ETL 任務（每天凌晨跑） ──────────→ BigQuery
 (OLTP)              ├── Extract: 抽取昨天的訂單          (OLAP)
                     ├── Transform: 清洗、Join 維度表
                     └── Load: 寫入 fact_orders 表
```

優點：簡單可靠，轉換邏輯集中管理。
缺點：資料有延遲（最多 24 小時），不適合需要即時分析的場景。

CDC（Change Data Capture）

監聽 OLTP 資料庫的 WAL（Write-Ahead Log），把每一筆資料變更即時串流到 OLAP 系統。資料延遲可以壓縮到幾秒到幾分鐘。

```
PostgreSQL ──→ Debezium（讀 WAL） ────→ Kafka ────→ ClickHouse
（OLTP）          每次 INSERT/UPDATE/DELETE                （OLAP，近即時）
                   都會產生一個事件
```

Debezium 是最常用的開源 CDC 工具，支援 PostgreSQL、MySQL、MongoDB 等主流 OLTP 資料庫。它讀取資料庫的複製日誌，把每一個資料變更轉換成事件發佈到 Kafka。

優點：資料延遲低，能做近即時的分析。
缺點：基礎設施更複雜，需要管理 Kafka 管線。

ELT（Extract, Load, Transform）

現代雲端倉儲（BigQuery、Snowflake）的算力很強，越來越流行的做法是先把原始資料載入倉儲，再在倉儲內做轉換（用 SQL 或 dbt）。

```
PostgreSQL ──→ 直接載入原始資料 ──→ BigQuery
                                      └── dbt 在倉儲內做轉換
                                          ├── 清洗
                                          ├── JOIN 維度表
                                          └── 建立 Mart 層的聚合表
```

dbt（data build tool） 已經成為現代資料工程的標準工具——用 SQL 定義轉換邏輯，做版本控制，自動管理依賴關係和執行順序。

HTAP：現代的混合方案

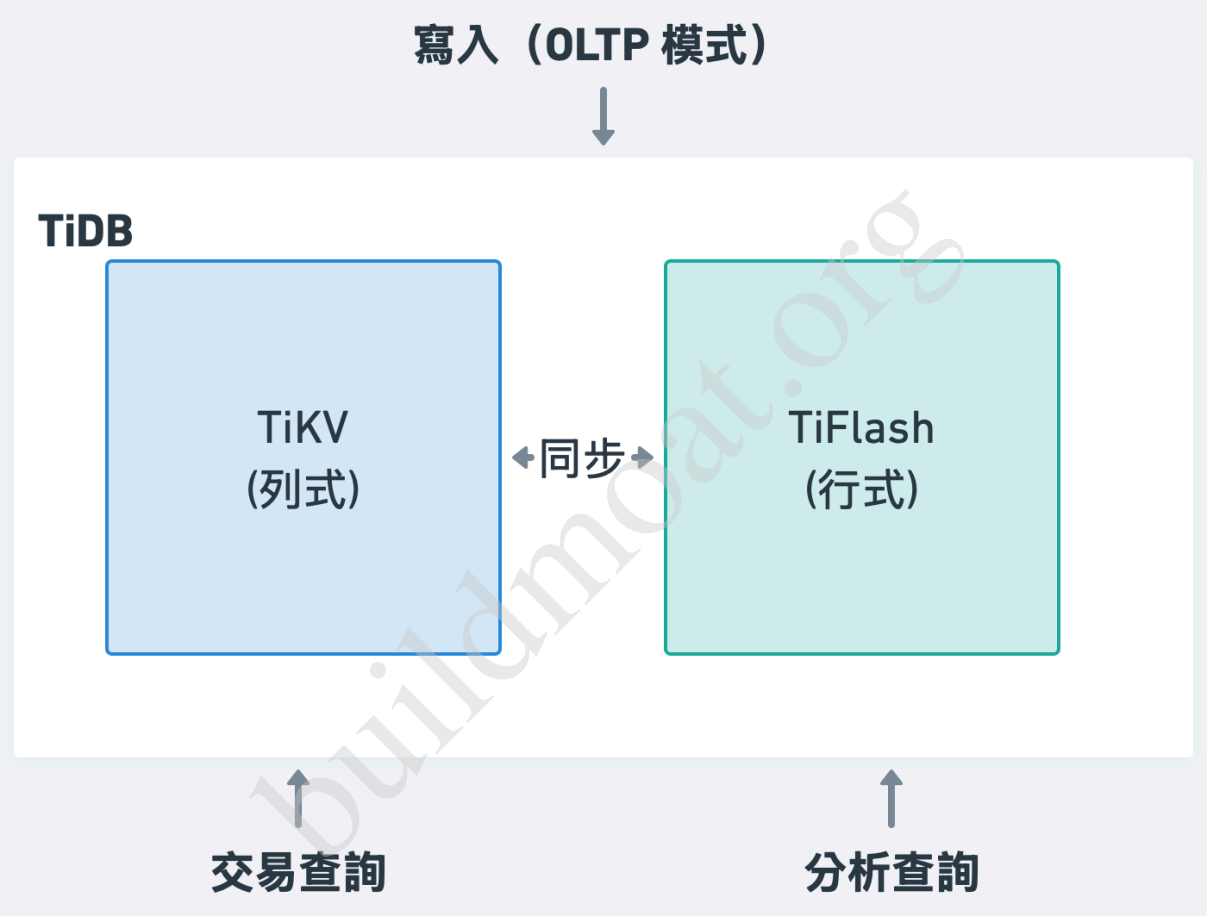
OLTP 和 OLAP 分開是因為兩者的底層設計衝突。但業界一直在嘗試用一個系統同時支援兩種工作負載，這就是 **HTAP（Hybrid Transactional/Analytical Processing）**。

buildmoat.org

TiDB：開源的分散式資料庫，內建了 TiKV（列式，處理 OLTP）和 TiFlash（行式，處理 OLAP）。同一份資料同時用兩種格式儲存，OLTP 寫入自動同步到行式副本，分析查詢走行式副本，互不干擾。

SingleStore（原 MemSQL）：行列混合儲存，在記憶體中保持列式格式（rowstore）處理交易，在磁碟上維持行式格式（columnstore）處理分析。

DuckDB：雖然主要是 OLAP 引擎，但它能直接查詢 PostgreSQL 的資料，在某些輕量場景下可以充當橋接層。



buildmoat.org

HTAP 的代價：同時維護兩種儲存格式，資源開銷更高，系統複雜度上升，很難做到對 OLTP 和 OLAP 都完全最佳化。對大多數公司，分開建置 OLTP + OLAP 還是更穩健的選擇。

什麼時候在面試裡用這些

系統裡出現分析需求時

任何涉及「報表」、「統計」、「儀表板」、「歷史資料分析」的需求，都是引入 OLAP 層的訊號。

「我們的 PostgreSQL 可以處理日常的業務操作，但財務報表需要跑複雜的聚合查詢，跑在同一個 PostgreSQL 上會影響線上業務的效能。我會引入一個 BigQuery 作為資料倉儲，透過 CDC + Kafka 把資料近即時地同步過去，讓分析查詢完全隔離在 BigQuery 上跑。」

設計資料管線時

當面試官問到「資料怎麼從業務系統流到分析系統」，這裡可以展開討論 ETL vs CDC vs ELT 的取捨：

「如果分析師能接受 T+1 的資料延遲，ETL 批次作業最簡單可靠。如果需要近即時的分析——例如即時的詐欺偵測或即時的運營儀表板——我會用 Debezium 讀 WAL，把變更事件發到 Kafka，再消費到 ClickHouse。這樣資料延遲可以控制在幾秒到幾分鐘。」

常見面試情境

設計 Uber 的分析系統：「行程資料寫入 PostgreSQL（OLTP，業務使用）。透過 CDC 把資料串流到 BigQuery（OLAP），駕駛的收入報表、城市的供需分析都在 BigQuery 上跑。需要即時監控的指標（例如目前各城市的在線司機數）走獨立的 Redis 計數器，不依賴 OLAP。」

設計電商的促銷系統：「訂單和庫存走 OLTP（DynamoDB）保證一致性和高並發。但促銷活動的效果分析（哪個 coupon 轉換率最高、哪個商品在促銷期間的銷售趨勢）需要掃描幾百萬筆訂單做聚合，這些放在 BigQuery 裡跑，透過每小時的 ETL 同步。」

常見的 Deep Dive 問題

「為什麼行式儲存對分析查詢更快？」

這是考察你是否理解底層原理，不只是記住結論。

「有三個原因疊加在一起。第一是 **I/O 效率**：分析查詢通常只需要幾個欄位，行式儲存只讀那幾個欄位的資料，跳過其他欄位；列式儲存必須讀取整行，大量 I/O 浪費在不需要的欄位上。第二是**壓縮**：同一欄位的值類型相同、值域相似，壓縮率遠高於混合不同類型的行；壓縮後資料量更小，讀取更快。第三是**向量化執行**：連續的同類型資料可以用 CPU 的 SIMD 指令做並行計算，一條指令同時處理幾十個值，大幅提升吞吐量。」

「什麼時候應該用 HTAP 而不是分開建 OLTP + OLAP？」

「HTAP 的吸引力在於：不需要維護資料管線，分析查詢能看到最新的資料（沒有 ETL 延遲），基礎設施更簡單。適合的場景是：資料量還不算極大（幾十 TB 以內）、對分析結果的即時性要求高、工程團隊資源有限。

buildroad.org

但大多數規模的公司，分開建置更務實。專門的 OLTP 和 OLAP 系統各自做到極致最佳化，HTAP 系統通常在兩邊都做了妥協。當資料量超過幾百 TB，或者 OLTP 和 OLAP 的負載模式差異很大，分開建置的效能和成本優勢就很明顯了。」

「ETL 和 CDC 怎麼選？」

「核心取捨是**延遲 vs 複雜度**。

ETL 批次作業邏輯簡單，錯了容易 debug，但資料有延遲——通常是幾小時到一天。如果業務分析師接受看昨天的資料，ETL 幾乎永遠是更好的選擇。

CDC 把延遲壓到幾秒，代價是引入了更多的基礎設施（Debezium、Kafka）和更多的故障點。Kafka consumer lag、schema 變更的處理、exactly-once 語義的保證——這些都是 CDC 管線需要處理的額外複雜度。

我的建議是：先用 ETL 起步，明確確認有即時分析的業務需求之後再引入 CDC。過早引入 CDC 的複雜度，很多時候是過度工程。」

總結

OLTP 和 OLAP 的區別，本質上是兩種不同的資料存取模式，推導出了兩種截然不同的儲存引擎設計：

OLTP：

- 列式儲存，讓點查詢和小量寫入高效
- B-tree 索引，讓查詢從 O(n) 變成 O(log n)
- ACID 事務，保證業務資料的一致性
- 設計目標：高併發、低延遲、強一致

OLAP：

- 行式儲存，讓全表掃描和聚合查詢只讀必要的欄位
- 高壓縮率，減少 I/O 量
- 向量化執行，充分利用 CPU 的並行能力
- 設計目標：高吞吐、大資料量、靈活查詢

連接兩者的管線：

- **ETL**：簡單可靠，有延遲，適合大多數場景
- **CDC**：近即時，基礎設施複雜，適合有明確即時需求的場景
- **ELT + dbt**：現代雲端倉儲的主流做法，把轉換邏輯留在倉儲內

buildmoat.org

在系統設計面試裡，看到任何涉及「統計」、「報表」、「分析」的需求，都應該主動說明你會把這類工作負載從 OLTP 分離到 OLAP——這展示了你理解不同工作負載的本質差異，而不是把所有東西都塞進同一個 PostgreSQL。

buildmoat.org